

INTRODUCTION TO LARAVEL

IT9a/L



What is Laravel?

Laravel is elegant, expressive and flexible **PHP framework** with extreme focus on clean code and speed which describes itself as “The PHP framework for web artisans”.

It is a free and Open-source PHP framework created by Taylor Otwell, based on **Model View Controller (MVC)** architecture.

Laravel framework provides various PHP libraries and helper functions and can help you to focus on more important pieces while providing common functions and logic to speed up the development time and ease up the development.



Why use Laravel?

1. MVC Architecture

- Laravel follows the **Model-View-Controller (MVC)** architectural pattern.
- This structural design separates your underlying business logic (Model) from your user interface (View), using the Controller to tie them together. This separation of concerns makes the code much easier to organize, maintain, and scale, especially when multiple developers are collaborating on the same project.



Why use Laravel?

2. Eloquent ORM (Object-Relational Mapping)

- Interacting with databases can often be tedious and messy, but **Laravel's Eloquent ORM** makes it incredibly intuitive. Instead of writing complex, raw SQL queries, developers can interact with the database using simple, object-oriented PHP syntax. It drastically speeds up database management and keeps the codebase highly readable.



Why use Laravel?

3. Rock-Solid Built-In Security

Security is a non-negotiable aspect of web development, and Laravel comes out of the box with robust protections against severe web vulnerabilities, including:

- **SQL Injection:** Prevented by default through PDO parameter binding.
- **Cross-Site Request Forgery (CSRF):** Laravel automatically generates security tokens for every active user session to prevent unauthorized commands.
- **Cross-Site Scripting (XSS):** Laravel automatically escapes output data, stopping malicious scripts from executing in the browser.



Why use Laravel?

4. Artisan Command-Line Interface (CLI)

- Laravel features its own built-in command-line tool called Artisan. It acts as an automated assistant that handles repetitive and tedious programming tasks. Developers can use Artisan to instantly generate boilerplate code, manage database migrations, and run custom scheduled tasks, which significantly accelerates the development timeline.



Why use Laravel?

5. Blade Templating Engine

- Blade is Laravel's lightweight yet powerful templating engine. It provides convenient shortcuts and structures for creating dynamic web pages (like template inheritance and conditional logic). Unlike some other engines, Blade compiles everything down to plain PHP code and caches it until it's modified, meaning it adds zero overhead to your application's performance.



Why use Laravel?

5. A Massive Ecosystem and Community

Laravel isn't just a standalone framework; it is an entire ecosystem equipped with officially supported tools for every stage of development and deployment:

- **Laravel Forge:** Simplifies server management and application deployment.
- **Laravel Vapor:** A serverless deployment platform for auto-scaling applications.
- **Laravel Echo:** Makes it easy to build real-time, interactive features (like live chat).



The MVC Architecture

Laravel applications follow the **Model-View-Controller** architecture design pattern. To understand Laravel, you must understand MVC. Imagine a restaurant:

- **The Model (The Kitchen/Pantry):** This is where your data lives. The Model handles all interactions with your database. It represents the entities in database and helps you to query the database and return the data.



The MVC Architecture

Laravel applications follow the **Model-View-Controller** architecture design pattern. To understand Laravel, you must understand MVC. Imagine a restaurant:

- **The View (The Plated Food):** This is what the user actually sees and interacts with—your HTML, CSS, and UI. These are the pages which will be displayed when accessed the app. View Component is used for the User Interface of the application.



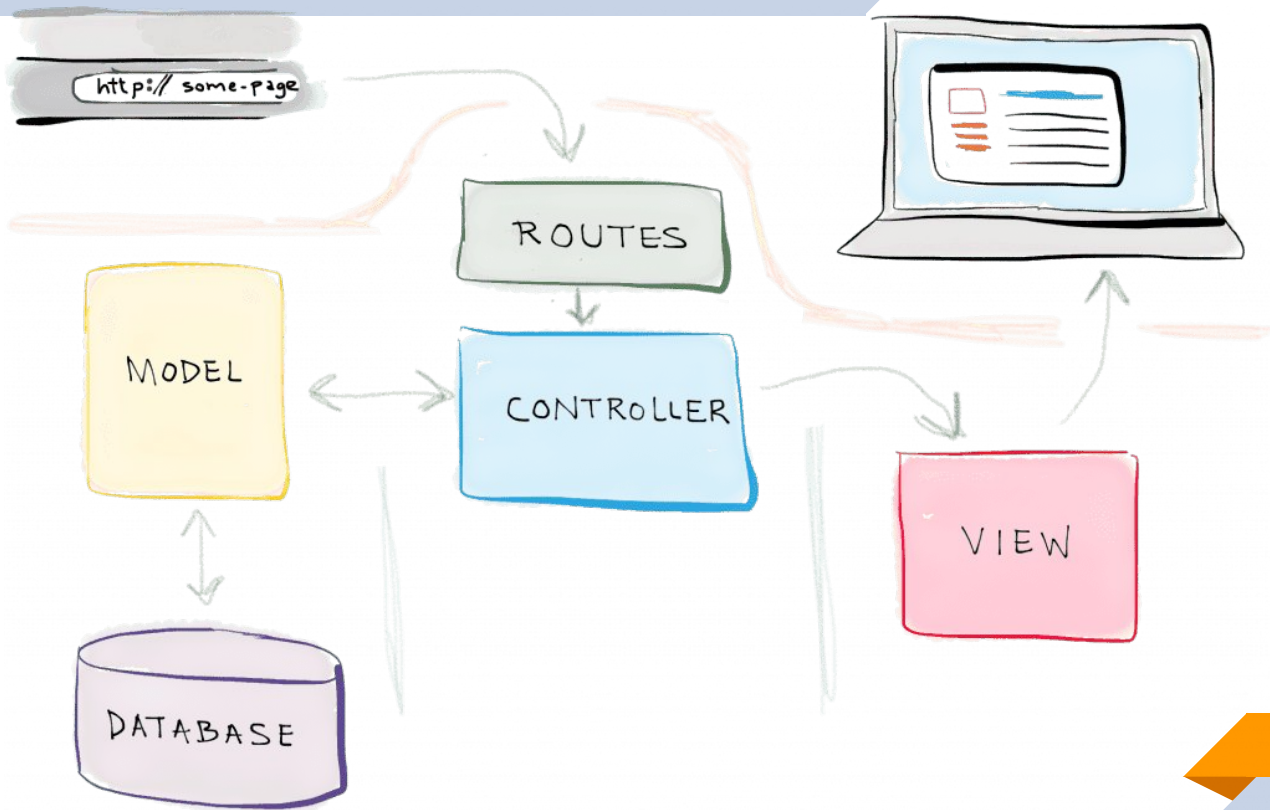
The MVC Architecture

Laravel applications follow the **Model-View-Controller** architecture design pattern. To understand Laravel, you must understand MVC. Imagine a restaurant:

- **The Controller (The Waiter):** It handle user requests, gets required data from the models and pass them to the Views. Controllers acts as an intermediary between Model and View Components to process the business logic and incoming request. The Controller takes the user's request (their order), goes to the Model (the kitchen) to get the required data (the food), and then passes it to the View (serves it to the table).



The MVC Architecture





Getting Started

We bring Laravel into existence using **Composer**, PHP's dependency manager. Once installed, we boot up our local environment using Laravel's built-in command-line tool, Artisan, by typing *php artisan serve*.



Download Composer Latest: v2.9.5



PRIVATE PACKAGIST

Secure & reliable package repository

Windows Installer

The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest composer version whenever it is executed.

Command-line installation

To quickly install Composer in the current directory, run the following script in your terminal. To automate the installation, use [the guide on installing Composer programmatically](#).

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') === 'c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eae
php composer-setup.php
php -r "unlink('composer-setup.php');"
```



Select Setup Install Mode



Select install mode

Composer can be installed for all users (requires administrative privileges), or for you only.



Install for all users (recommended)

→ Install for me only

Cancel



Composer Setup

Installation Options

Choose your installation type.



Setup will install Composer to a fixed location for all users. This includes a Control Panel uninstaller and is the recommended option. Click Next to use it.

☐ Developer mode

Take control and just install Composer. An uninstaller will not be included.

Next

Cancel



Composer Setup

Settings Check

We need to check your PHP and other settings.



Choose the command-line PHP you want to use:

C:\xampp\php\php.exe

Browse...

This is the PHP in your path. Click Next to use it.

Back

Next

Cancel

Composer Setup

Proxy Settings

Choose if you need to use a proxy.



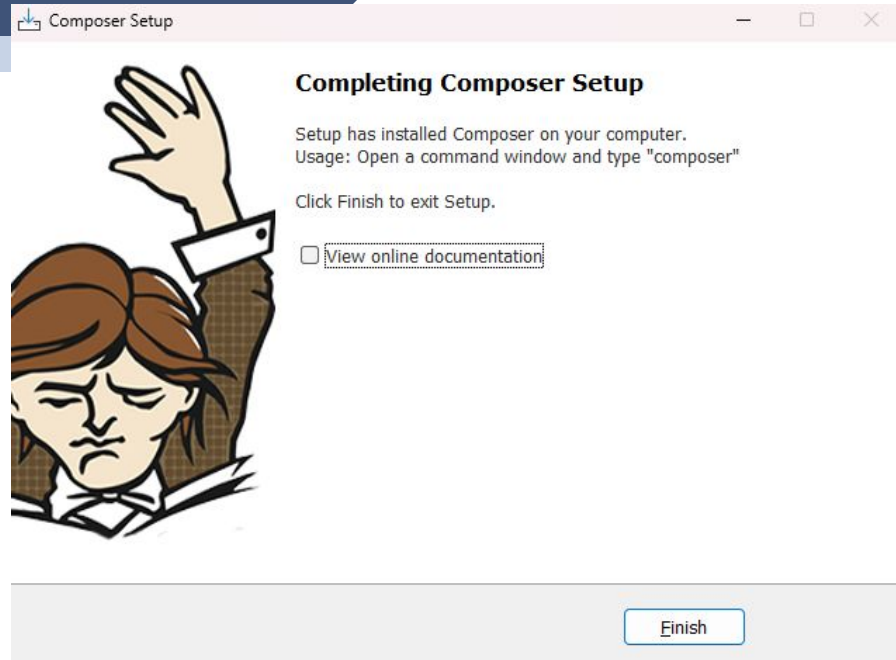
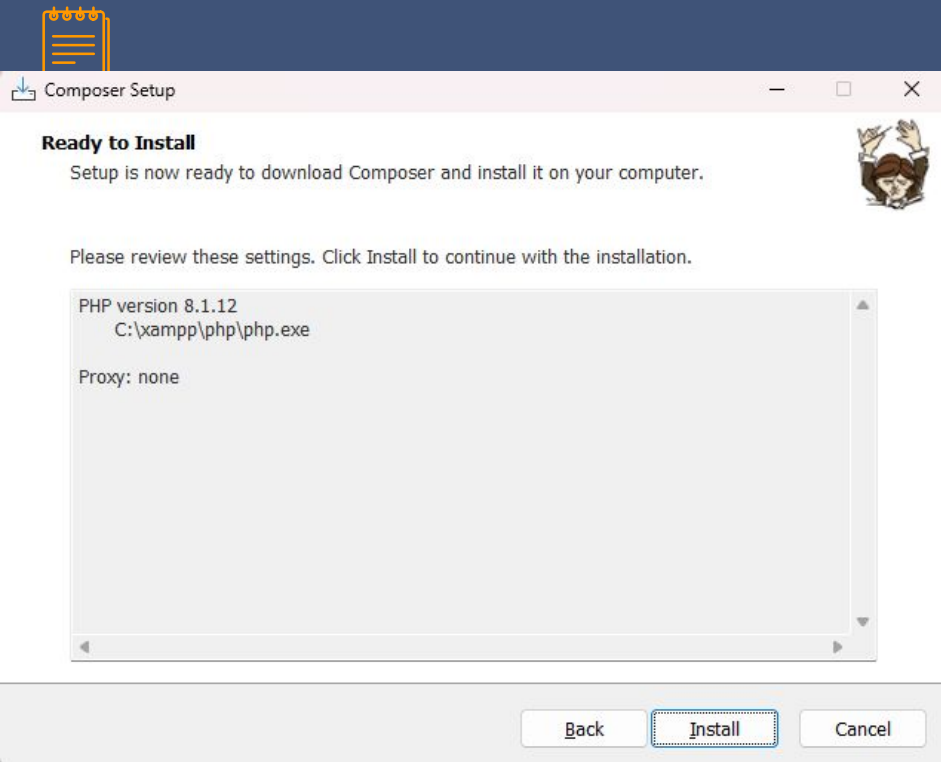
☐ Use a proxy server to connect to internet

Enter proxy url

Back

Next

Cancel



Installation

Meet Laravel

Laravel is a web application framework with expressive, elegant syntax. A web framework provides a structure and starting point for creating your application, allowing you to focus on creating something amazing while we sweat the details.

Laravel strives to provide an amazing developer experience while providing powerful features such as thorough dependency injection, an expressive database abstraction layer, queues and scheduled jobs, unit and integration testing, and more.

⌘K

After running one of the commands above, you should restart your terminal session. To update PHP, Composer, and the Laravel installer after installing them via `php.new`, you can re-run the command in your terminal.

If you already have PHP and Composer installed, you may install the Laravel installer via Composer:

```
1 composer global require laravel/installer
```



For a fully-featured, graphical PHP installation and management experience, check out [Laravel Herd](#).



File Edit Selection View ...

laravel

EXPLORER

LARAVEL

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR AZURE powershell

PS C:\Users\Kimberly_PC\Documents\laravel> composer global require laravel/installer

Changed current directory to C:/Users/Kimberly_PC/Appdata/roaming/composer

Cannot use laravel/installer's latest version v5.24.9 as it requires php ^8.2 which is not satisfied by your platform.

./composer.json has been created

Running composer update laravel/installer

Loading composer repositories with package information

Updating dependencies

Lock file operations: 28 installs, 0 updates, 0 removals

- Locking carbonphp/carbon-doctrine-types (3.2.0)
- Locking doctrine/inflector (2.1.0)
- Locking illuminate/collections (v10.49.0)

0 0 0

Signed out Go Live



File Edit Selection View ...

laravel

EXPLORER

▼ LARAVEL

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

SERIAL MONITOR

AZURE

powerShell +

Installing nesbot/carbon (2.73.0): Extracting archive

Installing illuminate/support (v10.49.0): Extracting archive

Installing symfony/finder (v6.4.34): Extracting archive

Installing illuminate/filesystem (v10.49.0): Extracting archive

Installing laravel/installer (v5.2.1): Extracting archive

18 package suggestions were added by new dependencies, use `composer suggest` to see details.

Generating autoload files

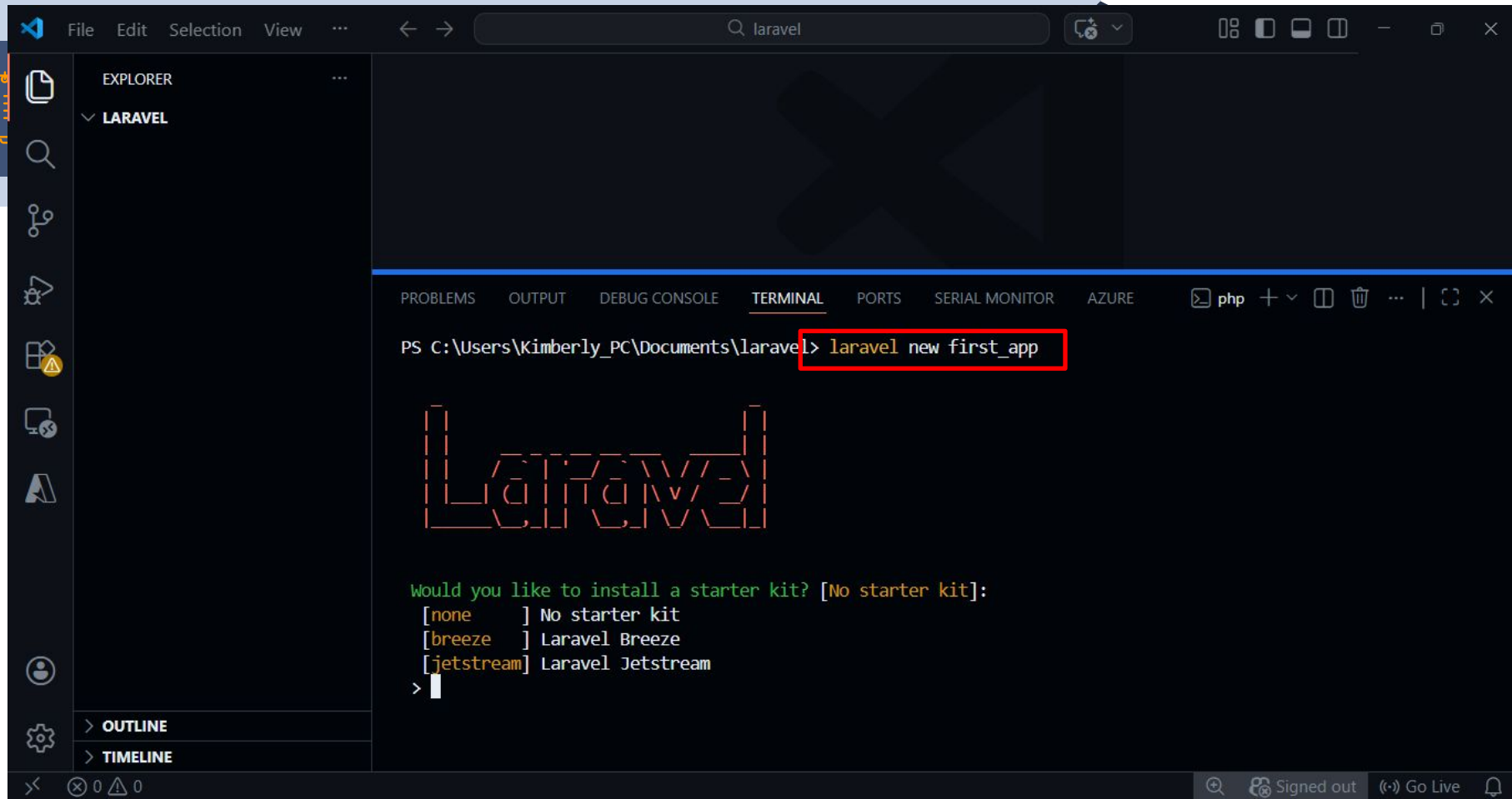
17 packages you are using are looking for funding.

Use the `composer fund` command to find out more!

No security vulnerability advisories found.

Using version ^5.2 for laravel/installer

PS C:\Users\Kimberly_PC\Documents\laravel>





```
C:\Users\Kimberly_PC\Documents>laravel new example-app
```

```
Would you like to install a starter kit? [No starter kit]:
```

```
[none    ] No starter kit
```

```
[breeze  ] Laravel Breeze
```

```
[jetstream] Laravel Jetstream
```

```
> none
```

```
Which testing framework do you prefer? [PHPUnit]:
```

```
[0] PHPUnit
```

```
[1] Pest
```

```
> 0
```

```
Would you like to initialize a Git repository? (yes/no) [no]:
```

```
> yes
```


File Edit Selection View Go Run

laravel

powerShell

EXPLORER

LARAVEL

first_app

app

bootstrap

config

database

node_modules

public

resources

routes

storage

tests

vendor

.editorconfig

.env

.env.example

.gitattributes

.gitignore

artisan

composer.json

composer.lock

package-lock.json

package.json

phpunit.xml

postcss.config.js

OUTLINE

TIMELINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

SERIAL MONITOR

AZURE

```
npm audit fix --force

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 8.19.2 -> 11.11.1
npm notice Changelog: <https://github.com/npm/cli/releases/tag/v11.11.1>
npm notice Run `npm install -g npm@11.11.1` to update!
npm notice

> build
> vite build

vite v5.4.21 building for production...
transforming...
✓ 54 modules transformed.
rendering chunks...
computing gzip size...
public/build/manifest.json 0.27 kB | gzip: 0.15 kB
public/build/assets/app-CKcAoPel.css 36.49 kB | gzip: 6.47 kB
public/build/assets/app-CWNHUVev.js 83.58 kB | gzip: 31.03 kB
✓ built in 2.14s

INFO Breeze scaffolding installed successfully.

warning: in the working copy of 'package.json', CRLF will be replaced by LF the next time Git touches it
INFO Application ready in [first_app]. Build something amazing.

PS C:\Users\Kimberly_PC\Documents\laravel>
```

24

Sign



Laravel Project Structure

1. ***app/*** (The Heart of Your Application)
2. ***routes/*** (The Traffic Cops)
3. ***resources/*** (Frontend Assets & Views)
4. ***database/*** (Data Structure & Dummy Data)
5. ***public/*** (The Web Server's Entry Point)
6. ***.env***
7. ***composer.json***

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```



Laravel Project Structure

1. *app/* (The Heart of Your Application)

- **Models (*app/Models/*):** This is the "**Model**" in MVC. These files represent your database tables. If you have a *users* table in your database, you will have a *User.php* model here. This model handles fetching, saving, and updating data for that specific entity.
- **Controllers (*app/Http/Controllers/*):** This is the "**Controller**" in MVC. Controllers act as the glue between your Models and your Views. When a user requests a page, the controller gathers the necessary data from the Model and passes it to the View to be displayed.

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```



Laravel Project Structure

2. *routes/* (The Traffic Cops)

- **web.php:** This file handles routes intended for web browsers. It includes features essential for web browsing, like session state (keeping users logged in) and CSRF protection (preventing cross-site request forgery attacks).
- **api.php:** This file is used if you are building an API (e.g., a backend for a mobile app or a React/Vue frontend). These routes are "stateless," meaning they don't use sessions and typically rely on tokens for authentication.

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```



Laravel Project Structure

3. *resources/* (Frontend Assets & Views)

- **resources/views/:** In Laravel, views are typically built using the Blade templating engine. These are essentially HTML files infused with some PHP logic (like loops and if statements) that allow you to display the dynamic data passed down from the Controllers.

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```



Laravel Project Structure

4. *database/* (Data Structure & Dummy Data)

- **Migrations:** Think of migrations as version control for your database. Instead of manually creating tables in a database management tool, you write PHP code here to define your table structures. This ensures every developer on the team has the exact same database schema.
- **Seeders & Factories:** These tools allow you to quickly populate your database with "dummy" or testing data. Instead of manually typing in 50 fake users to test your layout, a seeder and factory can generate them instantly.

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```



Laravel Project Structure

5. *public/* (The Web Server's Entry Point)

- **The Entry Point:** When you deploy a Laravel app, you configure your web server (like Apache or Nginx) to point only to this `public/` directory, not the root of your project.
- **Security:** Because the web server only has access to the `public/` folder, all of your sensitive application code (in `app/`), your environment variables, and your database configurations remain completely hidden and secure from the outside world.
- **Assets:** This is also where you place your publicly accessible files, such as compiled CSS, JavaScript, and images.

```
myapp/
├── app/
│   ├── Http/Controllers/
│   └── Models/
├── routes/
│   ├── web.php
│   └── api.php
├── resources/views/
├── database/
├── public/
├── .env
└── composer.json
```



Laravel Project Structure

6. .env

- This is your environment configuration file. It holds sensitive data like your database passwords, email server credentials, and third-party API keys. Crucially, this file should never be committed to version control (like GitHub) to keep your secrets safe.

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```



Laravel Project Structure

7. **composer.json**

- Laravel relies on Composer, a dependency manager for PHP. This file lists all the external packages and libraries your application needs to function.

```
myapp/  
├── app/  
│   ├── Http/Controllers/  
│   └── Models/  
├── routes/  
│   ├── web.php  
│   └── api.php  
├── resources/views/  
├── database/  
├── public/  
├── .env  
└── composer.json
```